



Sentiment Analysis – A Comparative Study



Presentation By

Neerajdattu Dudam (101179017)

Teja Mallireddy(101180616)

Supervisor: Dr. Lina Chato

April 13th, 2025

Inspiration and Objective

- **Inspiration:** To deepen our understanding of Natural Language Processing by exploring various sentiment analysis techniques. Driven by the need to compare traditional and modern approaches for practical sentiment classification.
- **Objective:** To implement and evaluate lexicon-based, machine learning, and deep learning models for sentiment analysis. To develop a comparative analysis of diverse models and feature extraction techniques on balanced datasets.
- **Key Goals:**
 - Implement and assess lexicon-based and classical ML sentiment classifiers.
 - Build and evaluate deep learning models including CNNs, LSTMs, and transformers.
 - Explore the effect of different feature engineering techniques like TF-IDF, Word2Vec, and GloVe.
 - Provide a comprehensive performance comparison to guide future sentiment analysis approaches.

Presentation Outline

- ☐ Introduction
- ☐ Text Preprocessing
- ☐ Lexicon and Machine Learning
- ☐ Workflow
- ☐ Data
- ☐ Experiments
- ☐ Results and Conclusions
- ☐ Future Directions
- ☐ References

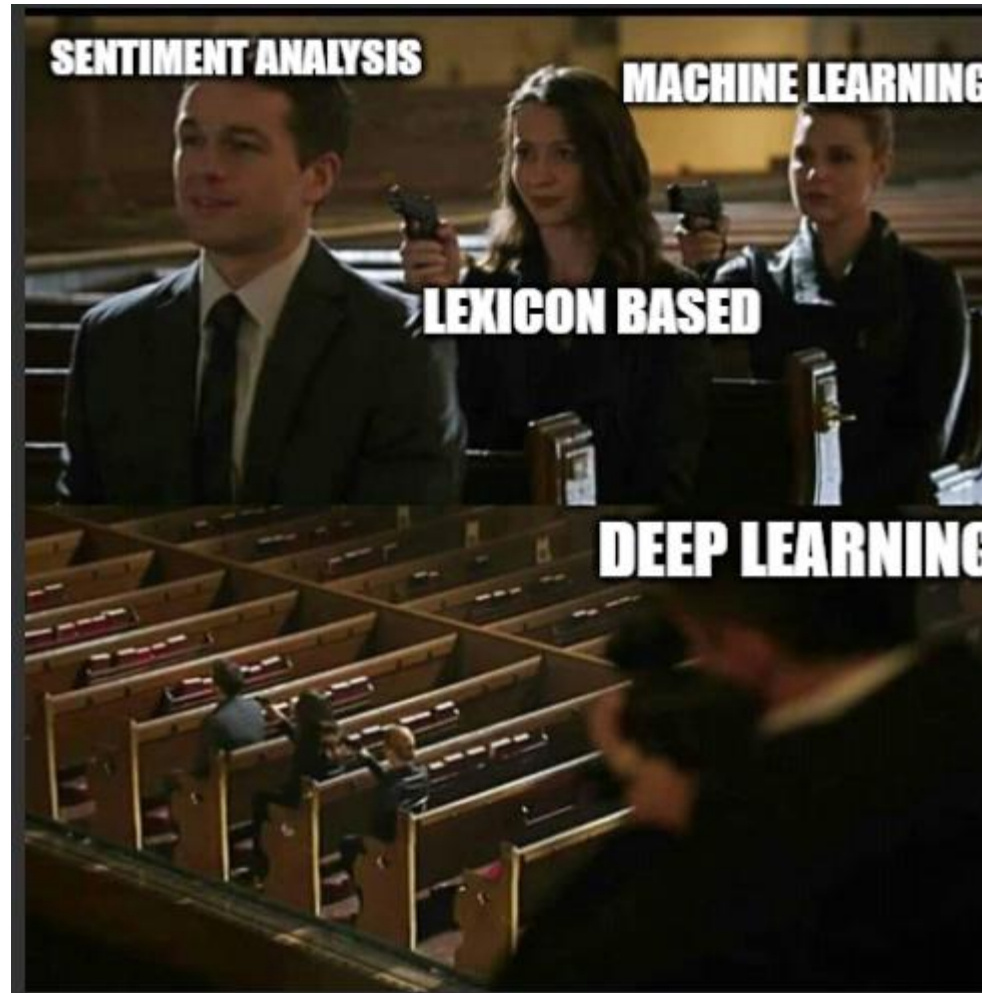
Introduction

- **Explosion of Textual Data:** In today's digital age, massive amounts of text are generated daily from social media, reviews, blogs, and feedback systems, making manual analysis impractical.
- **Importance of Sentiment Analysis:** A key NLP technique, sentiment analysis helps automatically determine the emotional tone—positive, negative, or neutral—embedded in textual content.
- **Applications Across Domains:** Beyond education, sentiment analysis is used in business for customer reviews, in politics for public opinion mining, and in healthcare for analyzing patient feedback.
- **Evolution of Techniques:** The field has evolved from lexicon-based methods like VADER to machine learning models such as Naive Bayes and SVMs, and now to advanced deep learning and transformer-based models like BERT and GPT.

Text Preprocessing

- **Level 1:** Tokenization, Stop words, Lemmatization
- **Level 2:** Bag of Words, TF IDF, Unigrams, Bigrams, N-Grams.
- **Level 3:** Word embeddings, word2vec, GloVe

Lexicon and Machine Learning



Lexicon and Machine Learning

➤ Lexicon Based

- Do not require training on labeled data.
- Rely on predefined sentiment lexicons where each word is associated with a sentiment score.
- Sentiment of a sentence or document is calculated based on the aggregate scores of the words it contains.
- Simple, interpretable, and useful for domains where labeled data is scarce.

VADER =>

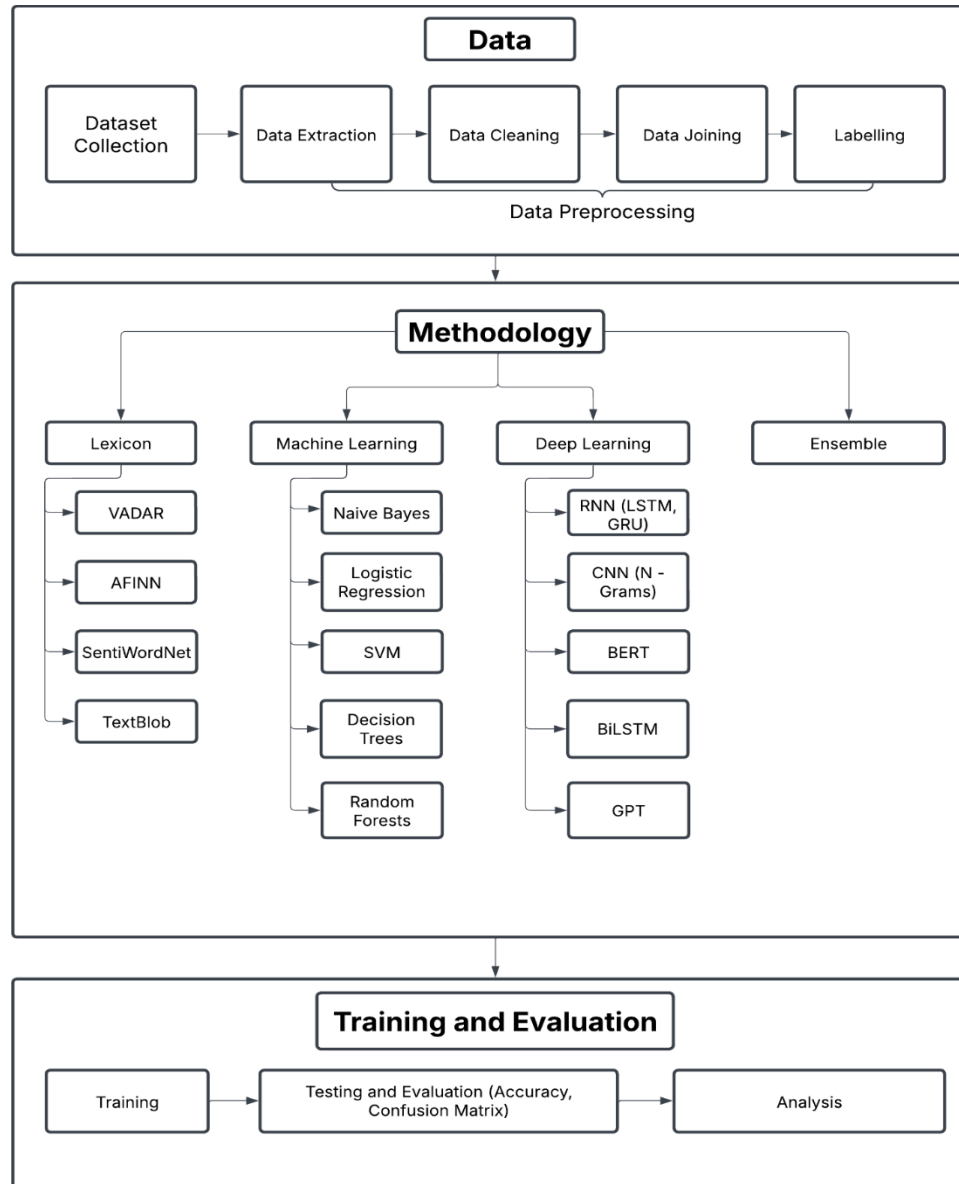
$$\text{Compound Score} = \frac{\sum_i s_i}{\sqrt{\sum_i s_i^2 + \alpha}}$$

Lexicon and Machine Learning

➤ Machine learning Based

- Require labeled data for training models to learn sentiment patterns.
- Convert text into numerical features using techniques like CountVectorizer or TF-IDF.
- Learn patterns and decision boundaries using algorithms like:
 - Naive Bayes
 - Logistic Regression
 - Support Vector Machines (SVM)
 - Decision Tree / Random Forest
- Can generalize better than lexicon methods on large, diverse datasets.
- More flexible and customizable, but may require more computational resources.

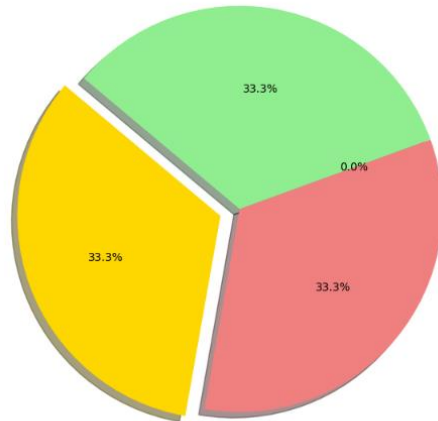
Workflow



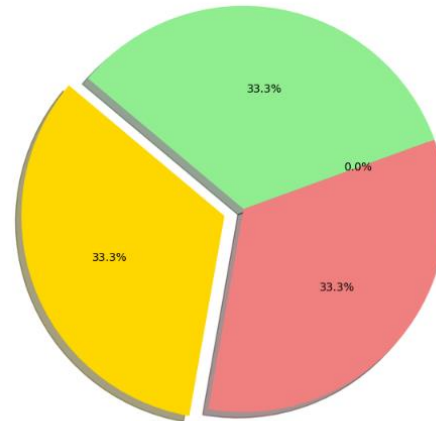
Data



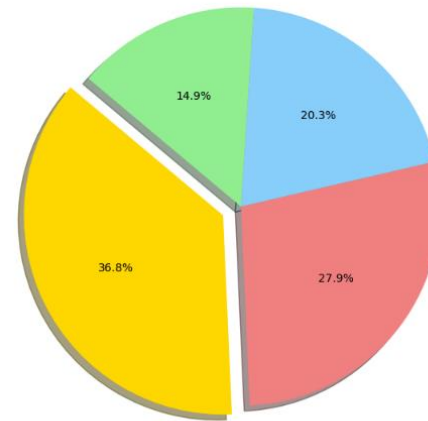
Distribution of Labels in dynasent-dynabench-dev.csv



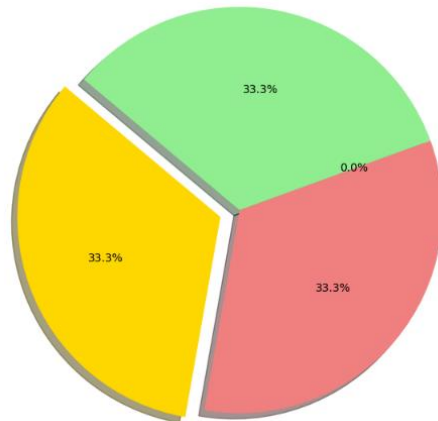
Distribution of Labels in dynasent-dynabench-test.csv



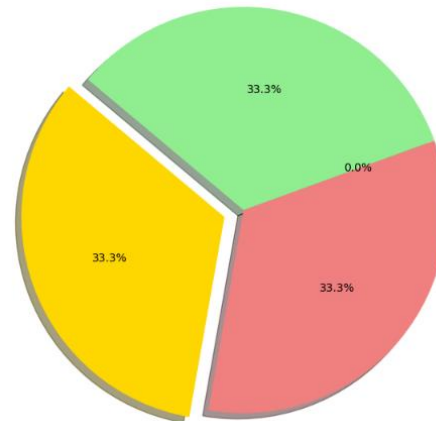
Distribution of Labels in dynasent-dynabench-train.csv



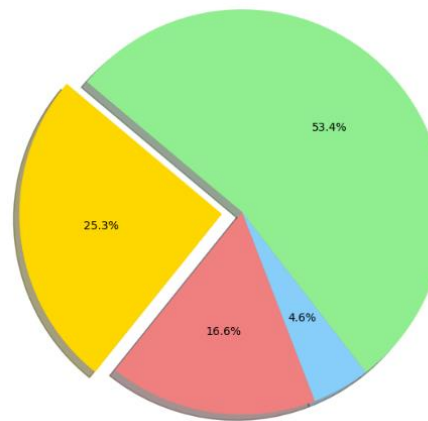
Distribution of Labels in dynasent-yelp-dev.csv



Distribution of Labels in dynasent-yelp-test.csv



Distribution of Labels in dynasent-yelp-train.csv



Data

file_name	total_rows	positive_rows	negative_rows	mixed_rows	neutral_rows
dynasent-dynabench-dev.csv	720	240	240	0	240
dynasent-dynabench-test.csv	720	240	240	0	240
dynasent-dynabench-train.csv	16399	6038	4579	3334	2448
dynasent-yelp-dev.csv	3600	1200	1200	0	1200
dynasent-yelp-test.csv	3600	1200	1200	0	1200
dynasent-yelp-train.csv	84388	21391	14021	3900	45076

File_name	Total_rows	Positive	Negative	Neutral
Yelp_train	42063	14021	14021	14021
Yelp_test	3600	1200	1200	1200
Yelp_val	3600	1200	1200	1200
dynabench_train	7344	2448	2448	2448
dynabench_test	720	240	240	240
dynabench_val	720	240	240	240
combined_train	49407	16469	16469	16469
combined_test	1440	480	480	480
combined_val	1440	480	480	480

Experiments

➤ **Exp 1: Using VADER**

$$\text{Compound Score} = \frac{\sum_i s_i}{\sqrt{\sum_i s_i^2 + \alpha}}$$

➤ **Exp 2: Using AFINN**

$$\text{Sentiment Score} = \sum_{i=1}^n \text{score}(w_i)$$

➤ **Exp 3: Using SentiWordNet**

$$\text{Sentiment Score}(w) = \frac{1}{N} \sum_{i=1}^N (\text{Pos}_i(w) - \text{Neg}_i(w))$$

➤ **Exp 4: Using TextBlob**

Experiments

➤ **Exp 5, 6: Naïve Bayes**

$$P(c, d) = \frac{P(c) \prod_{i=1}^n P(w_i|c)}{P(d)}$$
$$\hat{y} = \operatorname{argmax}_c (\log P(c) + \sum_{i=1}^n \log P(w_i|c))$$
$$P(w_i|c) = \frac{N_{w_i,c} + 1}{\sum_j N_{w_j,c} + |V|}$$

➤ **Exp 7,8,9: Logistic Regression**

➤ **Exp 10: SVM**

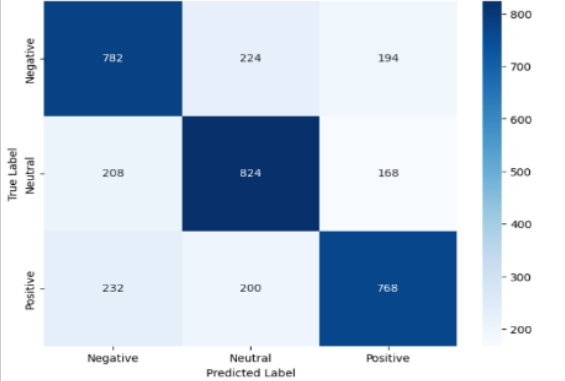
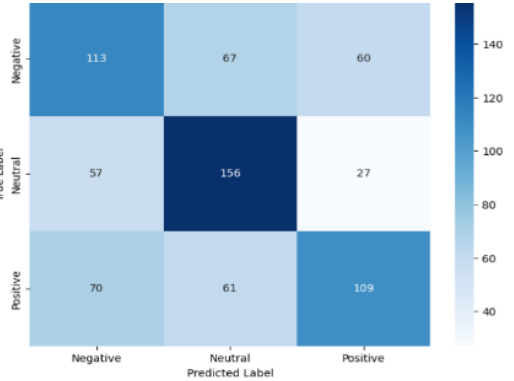
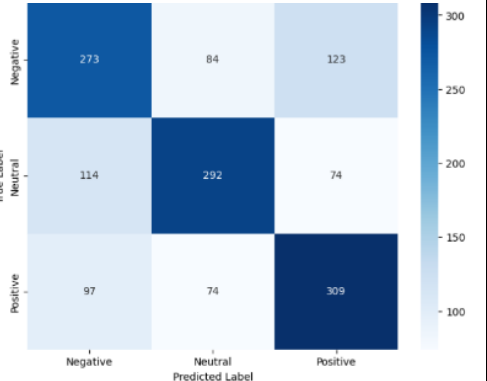
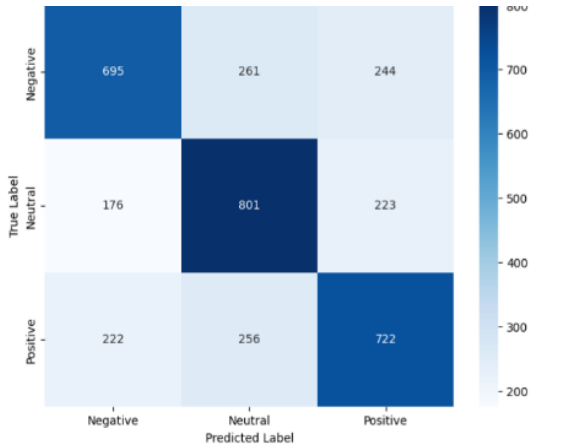
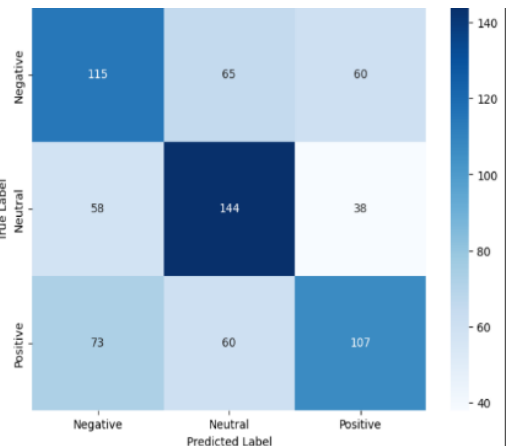
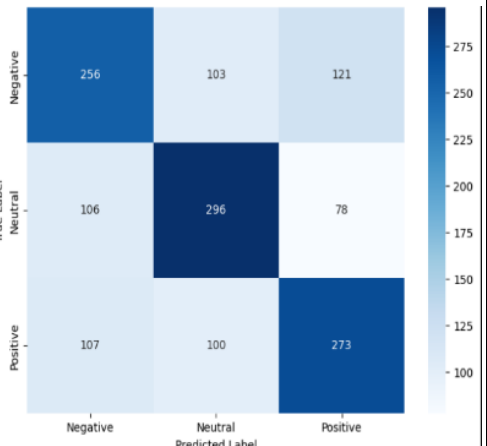
➤ **Exp 11: Decision Tree**

➤ **Exp 12: Random Forest**

Results

Exp No.	Experiment Name	Yelp Dataset	Dynabench Dataset	Combined Dataset
1	Using VADER	53	57	53
2	Using AFINN	52	57	53
3	Using SentiWordNet	43	44	43
4	Using TextBlob	50	54	51
5	Naive Bayes from Scratch	64	52	57
6	Multinomial Naive Bayes	65	53	57
7	Logistic Regression 1	54	49	45
8	Logistic Regression 2	60	50	56
9	Logistic Regression 3	62	51	57
10	SVM	66	53	61
11	Decision Tree	55	49	52
12	Random Forest	62	54	59

Results

Exp Name	Yelp Dataset	Dynabench Dataset	Combined Dataset																																																
SVM	 Confusion matrix for SVM on Yelp Dataset. True labels: Negative, Neutral, Positive. Predicted labels: Negative, Neutral, Positive. Color scale: 200 to 800. <table border="1"> <thead> <tr> <th>True Label \ Predicted Label</th> <th>Negative</th> <th>Neutral</th> <th>Positive</th> </tr> </thead> <tbody> <tr> <th>Negative</th> <td>782</td> <td>224</td> <td>194</td> </tr> <tr> <th>Neutral</th> <td>208</td> <td>824</td> <td>168</td> </tr> <tr> <th>Positive</th> <td>232</td> <td>200</td> <td>768</td> </tr> </tbody> </table>	True Label \ Predicted Label	Negative	Neutral	Positive	Negative	782	224	194	Neutral	208	824	168	Positive	232	200	768	 Confusion matrix for SVM on Dynabench Dataset. True labels: Negative, Neutral, Positive. Predicted labels: Negative, Neutral, Positive. Color scale: 40 to 140. <table border="1"> <thead> <tr> <th>True Label \ Predicted Label</th> <th>Negative</th> <th>Neutral</th> <th>Positive</th> </tr> </thead> <tbody> <tr> <th>Negative</th> <td>113</td> <td>67</td> <td>60</td> </tr> <tr> <th>Neutral</th> <td>57</td> <td>156</td> <td>27</td> </tr> <tr> <th>Positive</th> <td>70</td> <td>61</td> <td>109</td> </tr> </tbody> </table>	True Label \ Predicted Label	Negative	Neutral	Positive	Negative	113	67	60	Neutral	57	156	27	Positive	70	61	109	 Confusion matrix for SVM on Combined Dataset. True labels: Negative, Neutral, Positive. Predicted labels: Negative, Neutral, Positive. Color scale: 100 to 300. <table border="1"> <thead> <tr> <th>True Label \ Predicted Label</th> <th>Negative</th> <th>Neutral</th> <th>Positive</th> </tr> </thead> <tbody> <tr> <th>Negative</th> <td>273</td> <td>84</td> <td>123</td> </tr> <tr> <th>Neutral</th> <td>114</td> <td>292</td> <td>74</td> </tr> <tr> <th>Positive</th> <td>97</td> <td>74</td> <td>309</td> </tr> </tbody> </table>	True Label \ Predicted Label	Negative	Neutral	Positive	Negative	273	84	123	Neutral	114	292	74	Positive	97	74	309
True Label \ Predicted Label	Negative	Neutral	Positive																																																
Negative	782	224	194																																																
Neutral	208	824	168																																																
Positive	232	200	768																																																
True Label \ Predicted Label	Negative	Neutral	Positive																																																
Negative	113	67	60																																																
Neutral	57	156	27																																																
Positive	70	61	109																																																
True Label \ Predicted Label	Negative	Neutral	Positive																																																
Negative	273	84	123																																																
Neutral	114	292	74																																																
Positive	97	74	309																																																
Logistic Regression with TF-IDf features	 Confusion matrix for Logistic Regression with TF-IDf on Yelp Dataset. True labels: Negative, Neutral, Positive. Predicted labels: Negative, Neutral, Positive. Color scale: 200 to 800. <table border="1"> <thead> <tr> <th>True Label \ Predicted Label</th> <th>Negative</th> <th>Neutral</th> <th>Positive</th> </tr> </thead> <tbody> <tr> <th>Negative</th> <td>695</td> <td>261</td> <td>244</td> </tr> <tr> <th>Neutral</th> <td>176</td> <td>801</td> <td>223</td> </tr> <tr> <th>Positive</th> <td>222</td> <td>256</td> <td>722</td> </tr> </tbody> </table>	True Label \ Predicted Label	Negative	Neutral	Positive	Negative	695	261	244	Neutral	176	801	223	Positive	222	256	722	 Confusion matrix for Logistic Regression with TF-IDf on Dynabench Dataset. True labels: Negative, Neutral, Positive. Predicted labels: Negative, Neutral, Positive. Color scale: 40 to 140. <table border="1"> <thead> <tr> <th>True Label \ Predicted Label</th> <th>Negative</th> <th>Neutral</th> <th>Positive</th> </tr> </thead> <tbody> <tr> <th>Negative</th> <td>115</td> <td>65</td> <td>60</td> </tr> <tr> <th>Neutral</th> <td>58</td> <td>144</td> <td>38</td> </tr> <tr> <th>Positive</th> <td>73</td> <td>60</td> <td>107</td> </tr> </tbody> </table>	True Label \ Predicted Label	Negative	Neutral	Positive	Negative	115	65	60	Neutral	58	144	38	Positive	73	60	107	 Confusion matrix for Logistic Regression with TF-IDf on Combined Dataset. True labels: Negative, Neutral, Positive. Predicted labels: Negative, Neutral, Positive. Color scale: 100 to 275. <table border="1"> <thead> <tr> <th>True Label \ Predicted Label</th> <th>Negative</th> <th>Neutral</th> <th>Positive</th> </tr> </thead> <tbody> <tr> <th>Negative</th> <td>256</td> <td>103</td> <td>121</td> </tr> <tr> <th>Neutral</th> <td>106</td> <td>296</td> <td>78</td> </tr> <tr> <th>Positive</th> <td>107</td> <td>100</td> <td>273</td> </tr> </tbody> </table>	True Label \ Predicted Label	Negative	Neutral	Positive	Negative	256	103	121	Neutral	106	296	78	Positive	107	100	273
True Label \ Predicted Label	Negative	Neutral	Positive																																																
Negative	695	261	244																																																
Neutral	176	801	223																																																
Positive	222	256	722																																																
True Label \ Predicted Label	Negative	Neutral	Positive																																																
Negative	115	65	60																																																
Neutral	58	144	38																																																
Positive	73	60	107																																																
True Label \ Predicted Label	Negative	Neutral	Positive																																																
Negative	256	103	121																																																
Neutral	106	296	78																																																
Positive	107	100	273																																																

Conclusion and Future Directions

- Among the machine learning models, Logistic Regression and SVM with TF-IDF feature extraction showed better performance in sentiment classification.
- Lexicon-based methods provided quick and interpretable results but lacked contextual understanding.
- Traditional machine learning models performed well with proper preprocessing and feature engineering.
- Explore deep learning approaches such as CNN, RNN, LSTM, and transformer-based models like BERT. These models can better capture semantic context, word dependencies, and long-range relationships in text. Fine-tuning pretrained models like BERT on domain-specific data may further improve accuracy.

